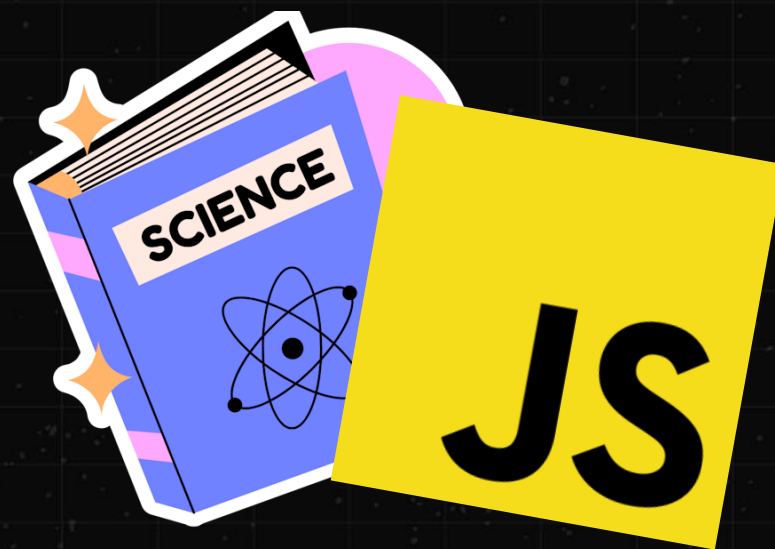


ЛЕКЦИЯ #19

CS BO FRONTEND



ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- * Поговорить про **операции поиска подстрок в строке**
- * Познакомимся с новой структурой данных — **префиксным деревом (бор, Trie)** для поиска сразу нескольких подстрок в строке
- * Разберём **понятие конечного автомата** и как это помогает в программировании

СТРОКА – ЭТО СТРУКТУРА ДАННЫХ

- * А значит, у нас есть **стандартный набор операций поиска и изменения**
- * Например, найти определённую подстроку в исходной строке
- * Или проверить, что строка соответствует заданным критериям
- * Или изменить что-то в строке
- * Однако, поскольку в **JavaScript строки неизменяемы**, фактически мы будем создавать новые строки*

А ПОЧЕМУ СО *?



ЕСЛИ СТРОКИ НЕИЗМЕНЯЕМЫЕ, ТО ...

- * V8, например, **оптимизирует многие операции со строками**
- * Если строка состоит только из символов Latin-1, то можно хранить её как однобайтовую (1 байт на символ)
- * Кроме того, оптимизируются **операции конкатенации и создания срезов**
- * Вместо немедленного копирования V8 создаёт специальные структуры, которые **хранят ссылки на исходные строки**
- * Но это всё — специфичные для VM оптимизации, **на которые полагаться в коде не стоит**

А КАКАЯ **СЛОЖНОСТЬ**
ОПЕРАЦИЙ НАД СТРОКОЙ?



ТУТ НЕ ВСЕ ТАК ПРОСТО

- * Во-первых, это **зависит от способа кодирования**
- * А во-вторых, как правило, мы будем взаимодействовать **не с одним символом, а с последовательностью**
- * Например, найти подстроку «**лава**» в строке «**Мирославу** **слава**»
- * **Какой будет алгоритм** для такой операции?

ПРЯМОЙ ПОИСК

- * Алгоритм основан на том, что мы **идём посимвольно по подстроке и сравниваем** каждый её символ с символом строки, начиная с позиции S
- * Если какие-то из **символов не совпадают**, мы начинаем операцию заново, но увеличиваем S на 1
- * То есть при каждой неудаче мы **возвращаемся к началу подстроки и смещаем S на 1**
- * **Сложность алгоритма $O(n \cdot m)$** ,
где n — длина строки поиска, а m — длина подстроки


```
function find(text, pattern) {
  const n = text.length;
  const m = pattern.length;

  if (m === 0) return 0;
  if (n < m) return -1;

  for (let i = 0; i <= n - m; i++) {
    let match = true;

    for (let j = 0; j < m; j++) {
      if (text[i + j] !== pattern[j]) {
        match = false;
        break;
      }
    }

    if (match) return i;
  }

  return -1;
}

// Примеры использования
console.log(find("Мирославу слава", "лава")); // 11
console.log(find("Hello World", "World"));    // 6
console.log(find("Hello World", "xyz"));      // -1
console.log(find("", ""));                    // 0
console.log(find("abc", ""));                 // 0
```

НО ПОЧЕМУ БЫ ПРОСТО
НЕ ПРОДОЛЖАТЬ ПОИСК
С НЕУДАЧНОГО СИМВОЛА?



К СОЖАЛЕНИЮ, ТАК НЕЛЬЗЯ

- * Для ряда строк **это будет работать неверно**
- * Например, в строке «**лилилилась**» ищем «**лилила**»
- * Мы видим, что слово в строке есть, но **такой упрощённый алгоритм его пропустит**
- * Выходит, что оптимизировать никак нельзя?
- * Можно, но для этого **нужно использовать более хитрые алгоритмы**

АЛГОРИТМ КМП (КНУТ – МОРРИС – ПРАТТ)

- * Алгоритм КМП позволяет **искать подстроку в строке за линейное время** $O(n + m)$
- * Придуман советским ученым [Юрием Владимировичем Матиясевичем](#)
(и независимо – Кнутом, Моррисом и Праттом)
- * В отличие от наивного поиска, где мы начинаем поиск с $S += 1$,
здесь мы **продолжаем поиск с $I - \text{maxPrefix(pattern)}$**
- * Где I – текущая позиция в строке поиска

В ЧЕМ ТУТ ИДЕЯ

- * После неудачи не возвращаться к $S + 1$, а использовать уже проверенную информацию
- * Таким образом мы отсекаем заранее лишнюю работу
- * КМП — это «умный» прямой поиск, который не пересчитывает уже проверенные символы
- * Важной частью алгоритма является **вычисление префикс-функции**

ПРЕФИКС-ФУНКЦИЯ

- * Применяется к шаблону поиска и возвращает массив, где каждый элемент равен **длине максимального префикса, который совпадает с суффиксом в подстроке**
- * Например, для «абабв»

Индекс	Символ	Подстрока	Префикс = Суффикс	Длина (π)
0	а	«а»	нет	0
1	б	«аб»	нет	0
2	а	«аб а»	«а» = «а»	1
3	б	«аб а б»	«аб» = «аб»	2
4	в	«аб а б в»	нет	0

```
function computePrefix(pattern) {  
  const m = pattern.length;  
  const pi = new Array(m).fill(0);  
  
  for (let i = 1; i < m; i++) {  
    let j = pi[i - 1];  
  
    while (j > 0 && pattern[i] !== pattern[j]) {  
      j = pi[j - 1];  
    }  
  
    if (pattern[i] === pattern[j]) {  
      j++;  
    }  
  
    pi[i] = j;  
  }  
  
  return pi;  
}  
  
// [ 0, 0, 1, 2, 0 ]  
console.log(computePrefix("абабв"));
```

```
function kmpFind(text, pattern) {
  const n = text.length;
  const m = pattern.length;
  const pi = computePrefix(pattern);

  let j = 0; // Индекс в паттерне

  for (let i = 0; i < n; i++) {
    while (j > 0 && text[i] !== pattern[j]) {
      j = pi[j - 1]; // ← умный сдвиг!
    }

    if (text[i] === pattern[j]) {
      j++;
    }

    if (j === m) {
      return i - j + 1; // Нашли!
    }
  }

  return -1;
}

// 2
console.log(kmpFind("лилилилась", "лилила"));
```

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

БЕЗ ПАНИКИ

- ✱ Наша задача — **понять основную идею**, нюансы в коде не столь важны
- ✱ В действительности это не «алгоритм поиска подстроки в строке», а «**алгоритм поиска фрагмента массива в массиве**»
- ✱ Вместо символов у вас может быть **любая сравнимая структура данных**

НАПРИМЕР

- * Биоинформатика – последовательности ДНК
- * Массивы чисел – шаблоны в данных
- * Обработка сигналов – паттерны во временных рядах
- * Компьютерное зрение – шаблоны в пикселях
- * И многое другое!

А ЕСЛИ МЫ ИЩЕМ
НЕ КОНКРЕТНУЮ ПОДСТРОКУ,
А НЕКОТОРЫЙ НАБОР?

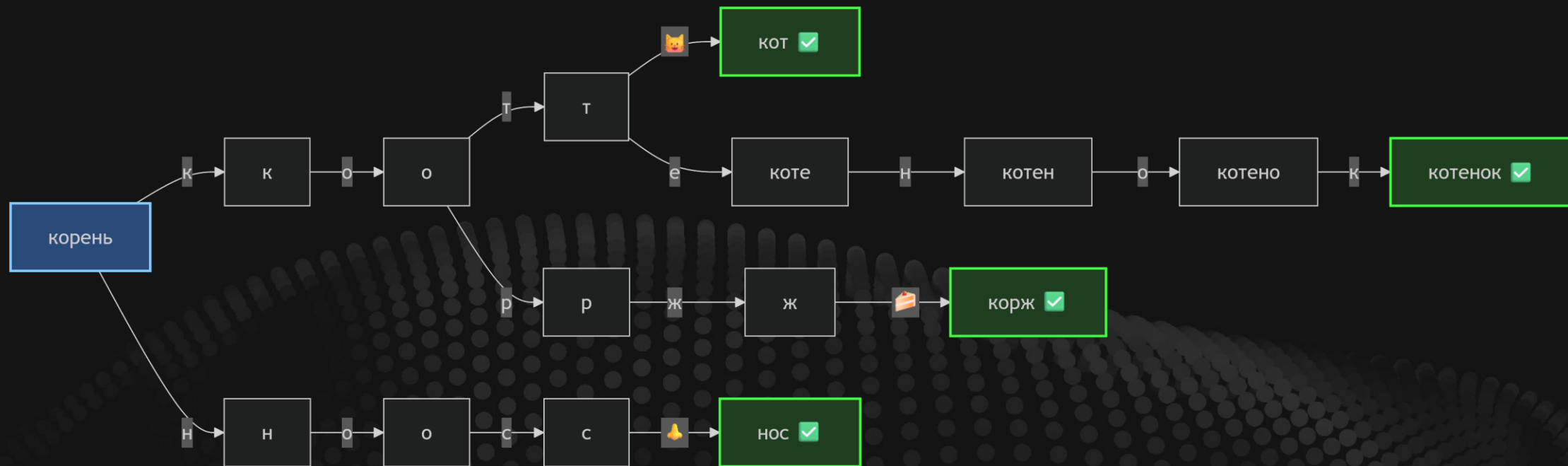


РЕШЕНИЕ «В ЛОБ»

- * То есть если поиск одной подстроки в строке алгоритмом КМП выполняется за линейное время $O(n + m)$
- * То поиск **нескольких подстрок** будет занимать $O(n \cdot k)$, где k — количество подстрок
- * Можно ли сделать это эффективнее?
- * Да, но для этого **нужно использовать новую структуру данных**

ВСТРЕЧАЙТЕ, ПРЕФИКСНОЕ ДЕРЕВО

- * Структура, также известная под именем «бор» или Trie
- * Представляет собой **корневое ориентированное дерево**
- * Корень символизирует «пустую строку»
- * Каждая **дуга взвешена буквой** (символом)
- * Если путь от корня до некоторой вершины формирует слово, то это **слово считается хранящимся в данной вершине**

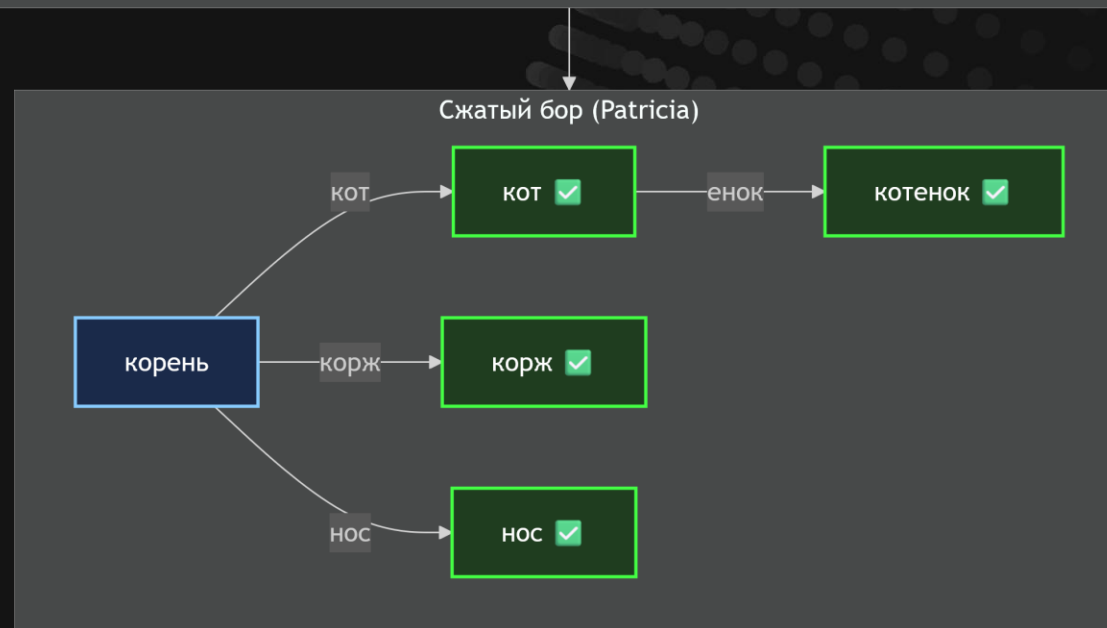
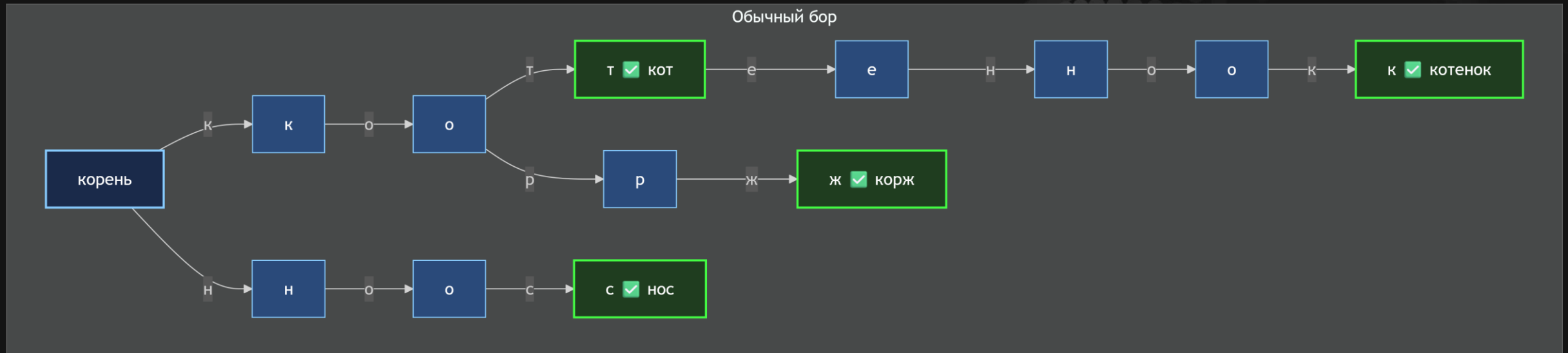


РЕФЛЕКСИРУЕМ

- * Мы построили бор для слов: «кот», «котенок», «корж», «нос»
- * Теперь, двигаясь по строке, мы **также двигаемся по бору**
- * Если из текущего узла **есть дуга, помеченная текущим символом строки**, то мы переходим по ней
- * Если **дуги нет, то нужно вернуться в корень** и повторить поиск со следующей позиции в строке

БОР МОЖНО «СЖАТЬ»

- * Сейчас **все символы имеют отдельные вершины**, даже если из вершины выходит только одна дуга
- * Такой бор **занимает больше памяти** и требует больше переходов
- * К счастью, бор можно сжать — вершины с одиночными связями «схлопываются» (получается сжатый бор или Patricia Trie)



РЕФЛЕКСИРУЕМ

- * Сжатый бор более **компактен по памяти и лучше кэшируется**
- * Однако алгоритм его реализации становится сложнее
- * Но зачастую он **работает эффективнее обычного бора**

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

МЫ УЖЕ ГОТОВЫ

- * Мы потратили много времени на изучение графов и деревьев, так что **новое дерево нас не испугает**
- * Разумеется, реализация такого дерева может отличаться в деталях
- * Но давайте **порerefлекслируем, что нам это дало**

ПРЯМОЙ ПОИСК ПЛОХ

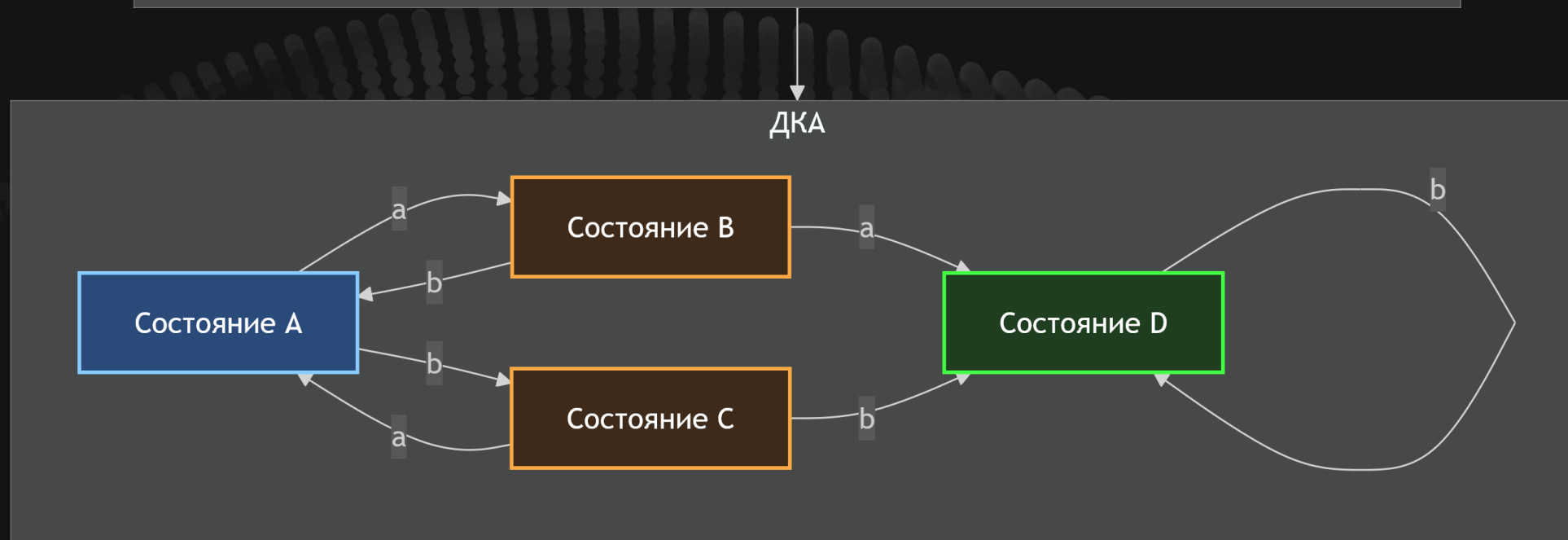
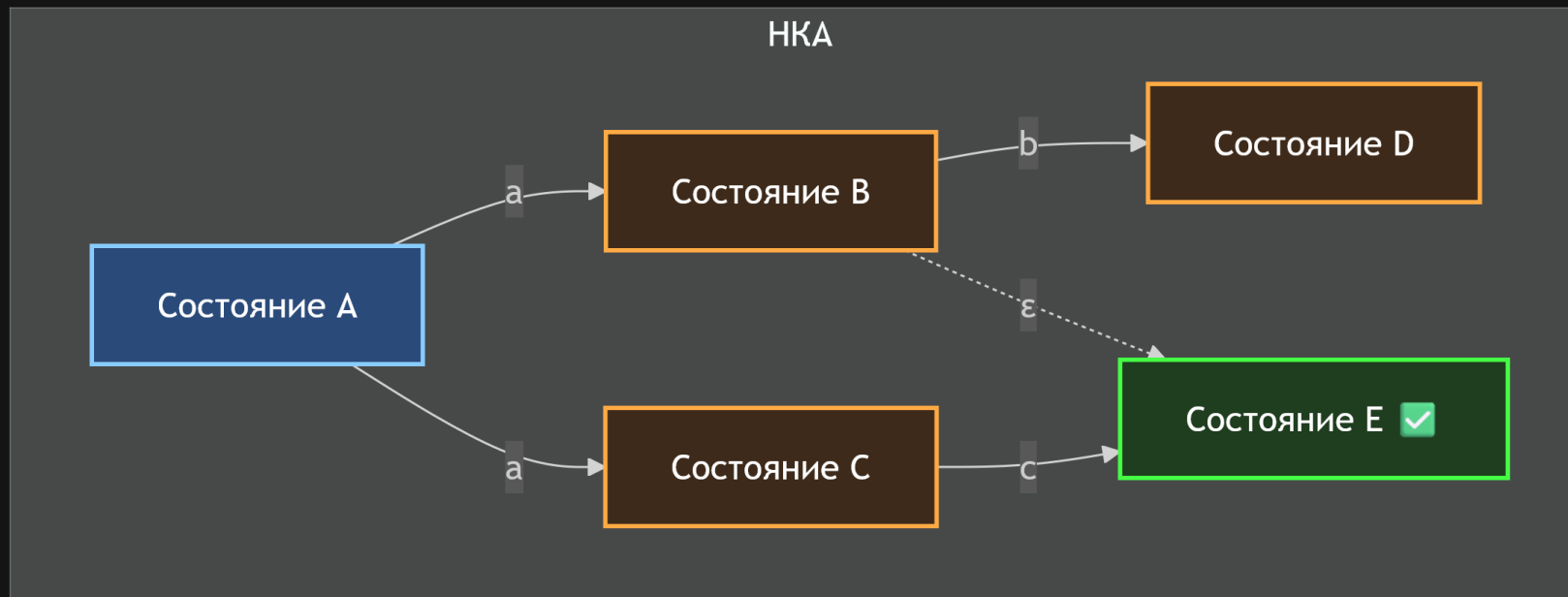
- * Да, мы можем искать сразу множество строк (или последовательностей), но **ценой квадратичной сложности** $O(n^2)$
- * С другой стороны, **алгоритм КМП может сделать поиск за линейное время** $O(n + m)$
- * Нужно придумать, как **адаптировать бор для поиска через КМП**
- * Так работает алгоритм «Ахо – Корасик»

НО ДЛЯ НАЧАЛА

- * Давайте познакомимся с термином «автомат» (state machine)
- * Это некоторое **устройство, у которого есть вход и выход**
- * **Некоторая «память»** (состояние)
- * Устройство может **переходить из одного состояния в другое** в зависимости от текущего состояния и входного сигнала
- * Если **из состояния нет переходов**, то оно называется тупиковым
- * А состояние, в котором автомат **завершает работу** (принимает вход), называется терминальным (принимающим)

КОНЕЧНЫЙ АВТОМАТ

- * Такой вид автомата, в котором количество возможных состояний фиксировано
- * Если все переходы из всех состояний однозначно определены, то такой автомат называется детерминированным
- * Для конечного автомата существуют два варианта:
ДКА (детерминированный) и **НКА** (недетерминированный конечный автомат)

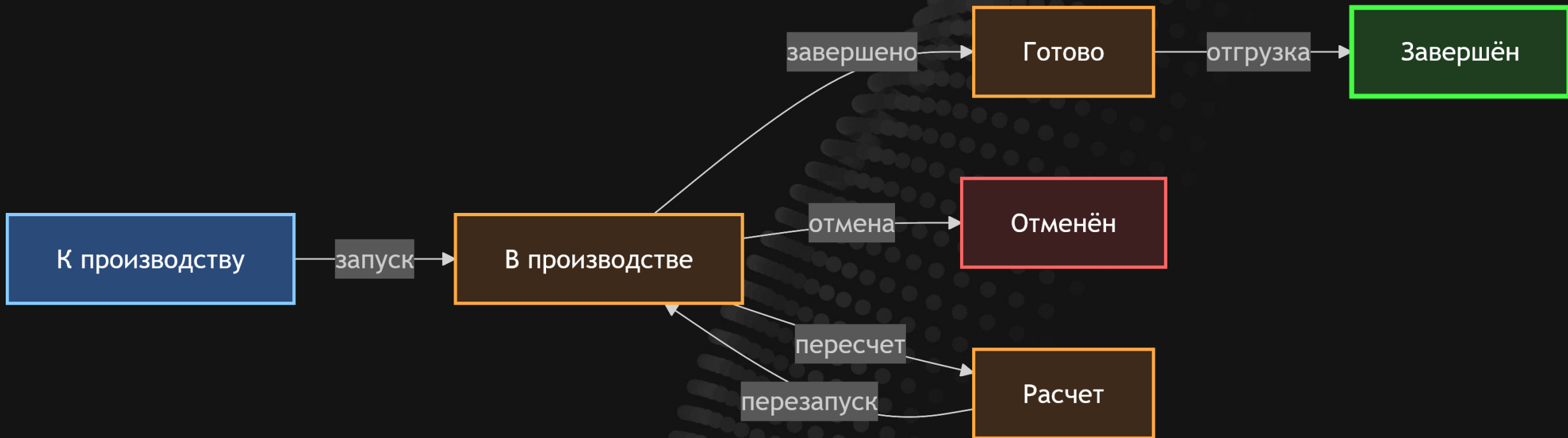


ТАК ЭТО
ГРАФ?



НУ, ЭТО СКОРЕЕ АБСТРАКЦИЯ

- * Да, конечный автомат **можно представить в виде графа**
- * Визуальное представление в виде графа особенно полезно **для отображения возможных состояний системы и переходов между ними**



А КАК ЭТО В КОДЕ
ПРЕДСТАВИТЬ?



ПО-РАЗНОМУ

- * Можно создать автомат с **графом состояний** и **отдельно функцию перехода** для перевода автомата в другое состояние
- * А можно просто завести переменную для состояния автомата и **всё сделать внутри самой функции перехода**
- * Выбор зависит от конкретной задачи и сложности автомата

```
// 1. ГРАФ + ФУНКЦИЯ ПЕРЕХОДА
const transitions = {
  "К производству": { "запуск": "В производстве" },
  "В производстве": { "завершено": "Готово", "отмена": "Отменён" },
  "Готово": { "отгрузка": "Завершён" }
};

function transition(state, event) {
  return transitions[state]?.[event] ?? state;
}

// 2. ВСЁ ВНУТРИ ФУНКЦИИ
let state = "К производству";

function handleEvent(event) {
  switch (state) {
    case "К производству":
      if (event === "запуск") state = "В производстве";
      break;
    case "В производстве":
      if (event === "завершено") state = "Готово";
      if (event === "отмена") state = "Отменён";
      break;
    case "Готово":
      if (event === "отгрузка") state = "Завершён";
      break;
  }
}
```

```

function parseNumber(str) {
  const states = {
    START: 0,
    SIGN: 1,
    INTEGER: 2,
    DECIMAL_POINT: 3,
    FRACTION: 4,
    DONE: 5,
    ERROR: 6,
  };

  let state = states.START;
  let result = "";
  let i = 0;
  const n = str.length;

  while (i < n && state !== states.ERROR && state !== states.DONE) {
    const ch = str[i];

    switch (state) {
      case states.START:
        if (ch === " " || ch === "\t" || ch === "\n") {
          i++;
          continue;
        } else if (ch === "-" || ch === "+") {
          result += ch;
          state = states.SIGN;
          i++;
        } else if (ch >= "0" && ch <= "9") {
          result += ch;
          state = states.INTEGER;
          i++;
        } else if (ch === ".") {
          result += "0.";
          state = states.FRACTION;
          i++;
        } else {
          state = states.ERROR;
        }
        break;

```

```

      case states.SIGN:
        if (ch >= "0" && ch <= "9") {
          result += ch;
          state = states.INTEGER;
          i++;
        } else if (ch === ".") {
          result += "0.";
          state = states.FRACTION;
          i++;
        } else {
          state = states.ERROR;
        }
        break;

      case states.INTEGER:
        if (ch >= "0" && ch <= "9") {
          result += ch;
          i++;
        } else if (ch === ".") {
          result += ch;
          state = states.DECIMAL_POINT;
          i++;
        } else {
          state = states.DONE;
        }
        break;

      case states.DECIMAL_POINT:
        if (ch >= "0" && ch <= "9") {
          result += ch;
          state = states.FRACTION;
          i++;
        } else {
          state = states.ERROR;
        }
        break;

```

```

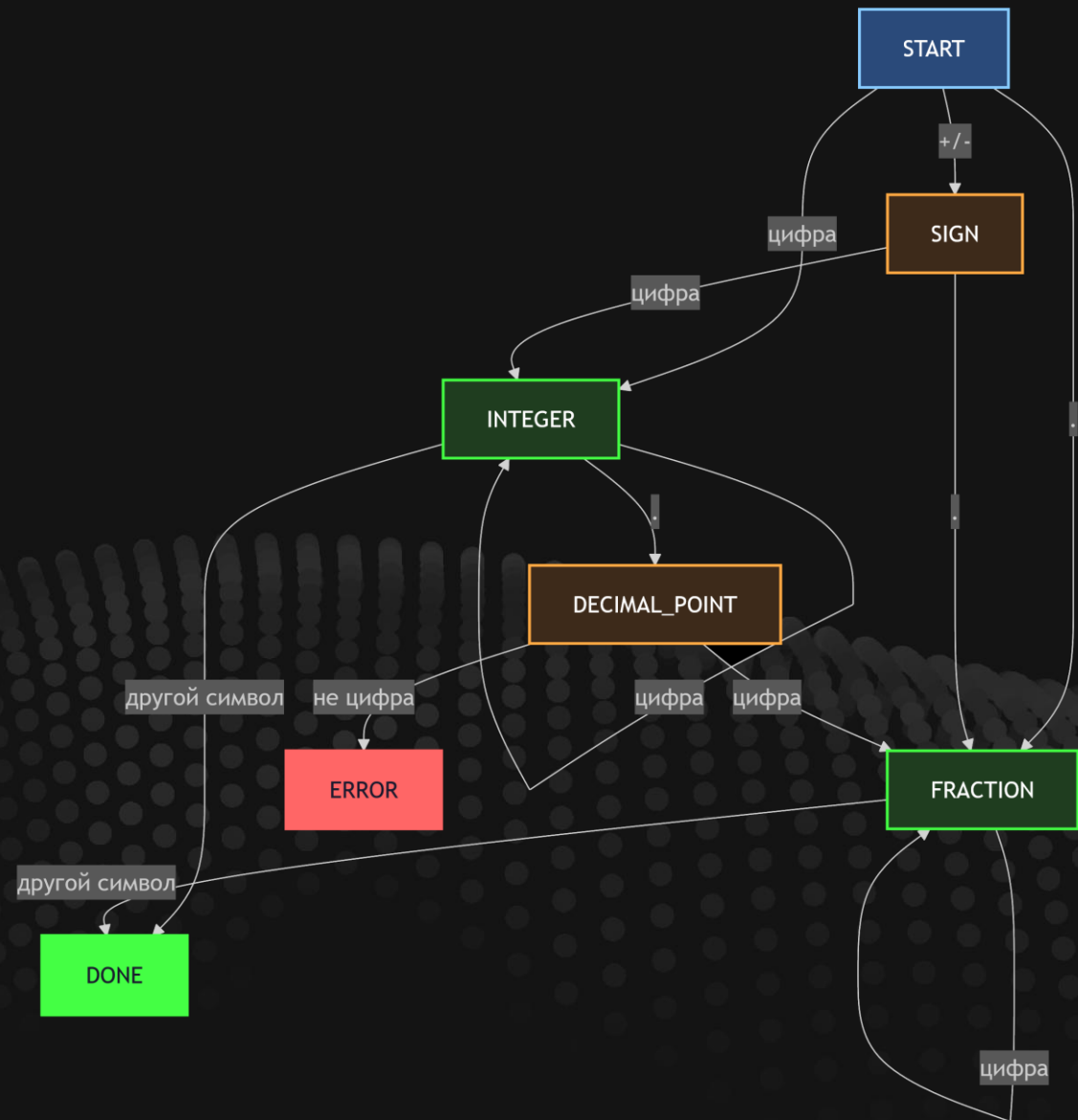
      case states.FRACTION:
        if (ch >= "0" && ch <= "9") {
          result += ch;
          i++;
        } else {
          state = states.DONE;
        }
        break;

      default:
        state = states.ERROR;
    }

    if (state === states.ERROR || result === "") {
      return NaN;
    }

    return Number(result);
  }
}

```



РЕФЛЕКСИРУЕМ

- * Строго говоря, **любая программа** — это реализация конечного автомата
- * Но при программировании нам **не всегда нужно использовать эту абстракцию** явно
- * Однако когда возникает **задача с чёткой последовательностью переходов**, использование конечного автомата становится крайне удобным

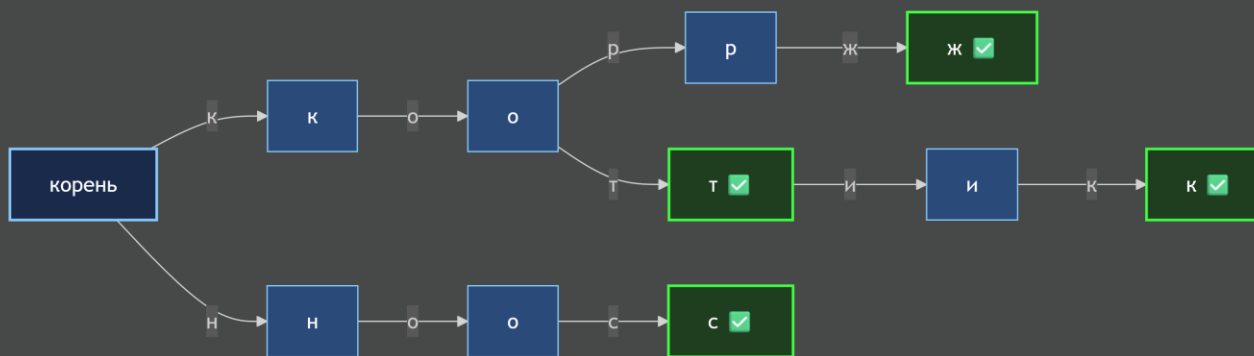
ТАК ЧТО ТАМ
С **ПОИСКОМ**
В СТРОКЕ?



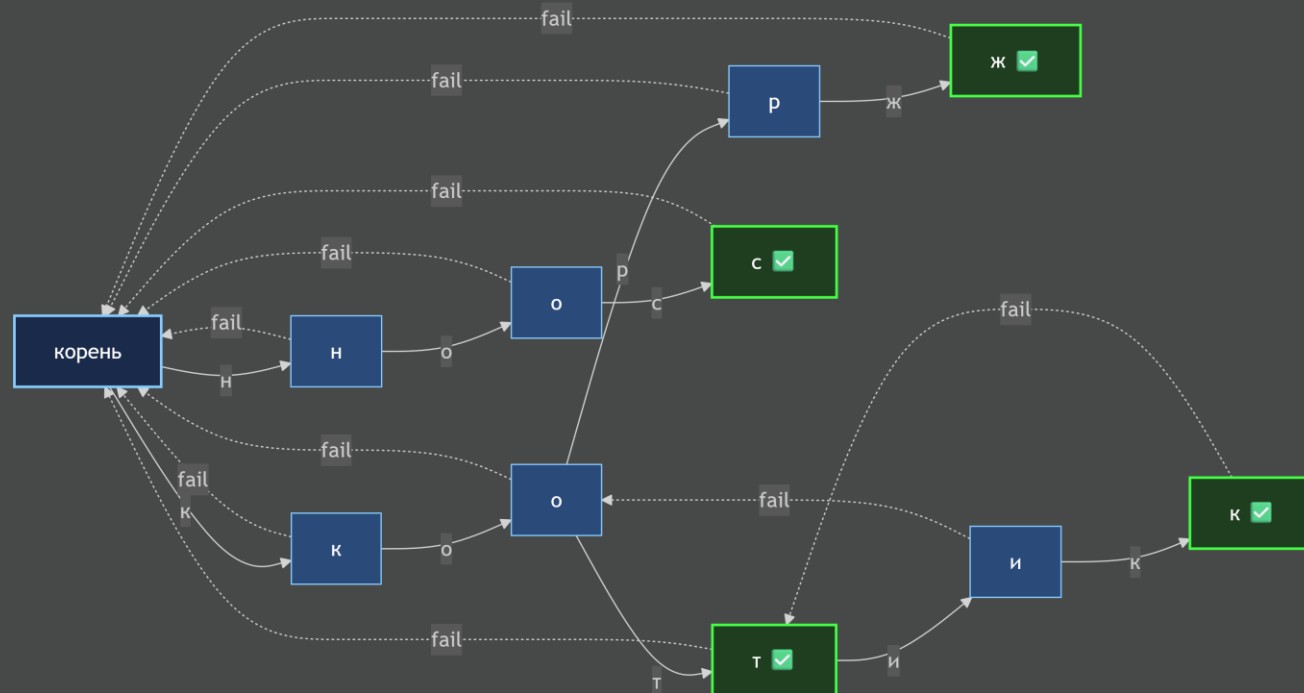
АХО – КОРАСИК

- * Один из популярнейших алгоритмов поиска подстрок в строке – алгоритм Ахо – Корасик
- * Он позволяет **искать сразу несколько подстрок за линейное время** $O(n + \text{сумма длин паттернов})$
- * Алгоритм основан на идее **построения конечного автомата на основе бора**
- * У каждого узла, кроме корня, есть **ссылка на самый длинный суффикс этого узла**, который также является префиксом одного из паттернов (fail-ссылка)
- * Строим конечный автомат на основе бора, где вершины бора **определяют состояние автомата, а дуги – переходы**

1. Строим бор



2. Добавляем fail-ссылки



second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

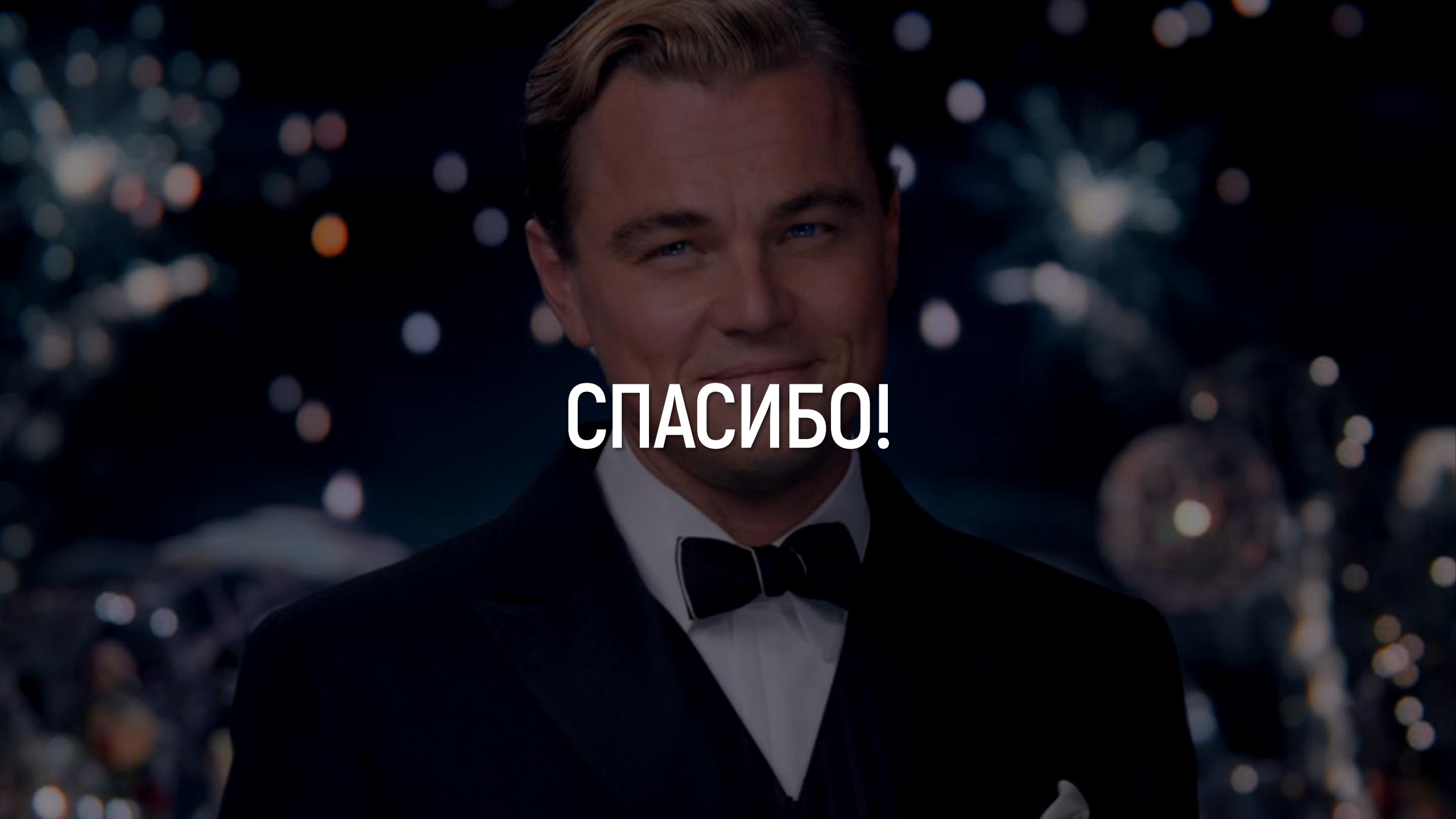
СТОЖКА, СТОЖКААА!

ВЫДЫХАЕМ

- * Суть алгоритма в том, что в случае неуспешного сравнения символов мы **восстанавливаемся до ближайшего состояния**, а не сбрасываемся в начало
- * Однако нужно понимать, что перед началом поиска **мы должны построить конечный автомат**
- * Это **требует времени и памяти**
- * Если мы ищем только одну подстроку, то **КМП будет эффективнее**
- * А вообще алгоритмов поиска подстрок существует множество
- * И держим в голове, что **строками алгоритм не ограничивается**

ПОДВЕДЁМ ИТОГИ

- * Алгоритмы над строками — это отдельная большая тема
- * Наша задача сводится к **пониманию проблемы и базовых приёмов решения**
- * Новый кейс использования деревьев
- * И, конечно же, знакомство с конечными автоматами
- * Если вам ничего не понятно, **то это нормально**
- * Рост там, где сложно

A close-up portrait of Leonardo DiCaprio, smiling slightly, wearing a dark tuxedo with a white shirt and a dark bow tie. The background is dark with numerous out-of-focus, colorful bokeh lights in shades of blue, white, and orange, suggesting a night-time event or fireworks.

СПАСИБО!